

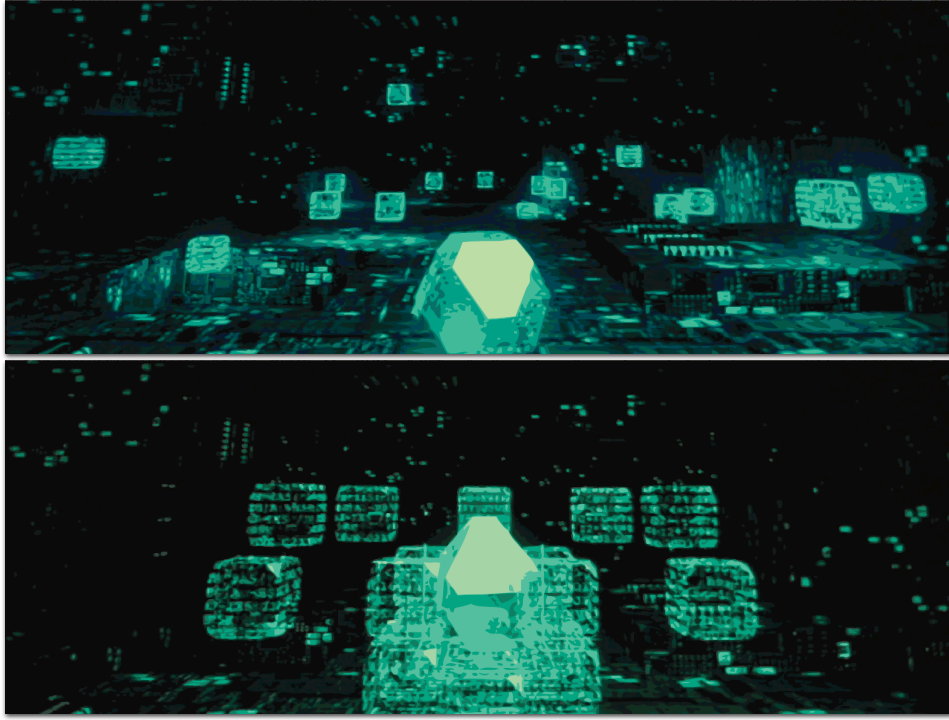


Figure 2: *Service-Oriented Architectures: Abstract visualisations, with the loosely joined services as cuboids, and the service orchestration as a polyhedron.*

to consist of such third-party services combined with others created in-house, which has the potential to spread costs over many users and uses, and promote standardisation both in and across industries. For example, the travel industry now has a well-defined, and documented, set of both services and data, sufficient to allow any competent software engineer to create travel agency software using entirely off-the-shelf software services [13]. Other industries, such as the finance industry, are also making significant progress in this direction [48].

The vision of SOAs assembling application components from a loosely coupled network of services, that can create dynamic business processes and agile applications that span organisations and computing platforms, is visualised in Figure 2. It will be made possible by creating compound solutions that use internal organisational software assets, including enterprise information and legacy systems, and combining these solutions with external components residing in remote networks. The great promise of SOAs is that the *marginal cost* of creating the n -th application is virtually zero, as all the software required already exists to satisfy the requirements of other applications. Only their *combination* and *orchestration* are required to produce a new application [33]. The *key* is that the interactions between the *chunks* are not specified within the *chunks* themselves. Instead, the interaction of services (all of whom are hosted by unassociated peers) is specified by users in an ad-hoc way, with the intent driven by newly emergent business requirements [27].

The pinnacle of SOA interoperability, is the exposing of services on the internet as *web services*. A web service is a specific type of service that is identified by a Uniform Resource Identifier (URI), whose service description and transport utilise open Internet standards. Interactions between web services typically occur as Simple Object Access Protocol (SOAP) calls carrying eXtensible Markup Language (XML) data content. The interface descriptions of web services are expressed using the Web Services Definition Language (WSDL) [36]. The Universal Description Discovery and Integration (UDDI) standard defines a protocol for

directory services that contain web service descriptions. UDDI enables web service clients to locate candidate services and discover their details. Service clients and service providers utilise these standards to perform the basic operations of SOAs [36]. Service aggregators can then use the Business Process Execution Language (BPEL) to create new web services by defining corresponding compositions of the interfaces and internal processes of existing services [36].

SOA services inter-operate based on a formal definition (or contract, e.g. WSDL) that is independent of the underlying platform and programming language. Service descriptions are used to advertise the service capabilities, interface, behaviour, and quality [36]. The publication of such information about available services provides the necessary means for discovery, selection, binding, and composition of services [36]. The (expected) behaviour of a service during its execution is described by its behavioural description (for example, as a workflow process). Also, included is a quality of service (QoS) description, which publishes important functional and non-functional service quality attributes, such as service metering and cost, performance metrics (response time, for instance), security attributes, integrity (transactional), reliability, scalability, and availability [36]. Service clients (end-user organisations that use some service) and service aggregators (organisations that consolidate multiple services into a new, single service offering) utilise *service descriptions* to achieve their objectives [36]. One of the most important and continuing developments in SOAs is Semantic Web Services (SWS), which make use of *semantic descriptions* for service discovery, so that a client can discover the services semantically [11].

There are multiple standards available and still being developed for SOAs, most notably of recent being REpresentational State Transfer (REST) [42]. The software industry now widely implements a thin SOAP/WSDL/UDDI veneer atop existing applications or components that implement the web services paradigm, but the choice of technologies will change with time. Therefore, the fundamentals of SOAs and its services are best defined generically, because SOAs are technology agnostic and need not be tied to a specific technology [37]. Within the current and future scope of the fundamentals of SOAs, there is clearly potential to *evolve* complex high-level software applications from the modular services of SOAs, instead of the instruction level evolution currently prevalent in genetic programming [21].

4 Distributed Evolutionary Computing

Having previously suggested evolutionary computing [10, 9], and the possibility of it occurring within a distributed environment, not unlike those found in mobile agent systems, leads us to consider a specialised form known as DEC. The fact that evolutionary computing manipulates a population of independent solutions actually makes it well suited for parallel and distributed computation architectures [12]. The motivation for using parallel or distributed evolutionary algorithms is twofold: first, improving the speed of evolutionary processes by conducting concurrent evaluations of individuals in a population; second, improving the problem-solving process by overcoming difficulties that face traditional evolutionary algorithms, such as maintaining diversity to avoid premature convergence [44]. There are several variants of distributed evolutionary computing, leading some to propose a taxonomy for their classification, with there being two main forms [12, 44]:

- multiple-population/coarse-grained migration/island
- single-population/fine-grained diffusion/neighbourhood

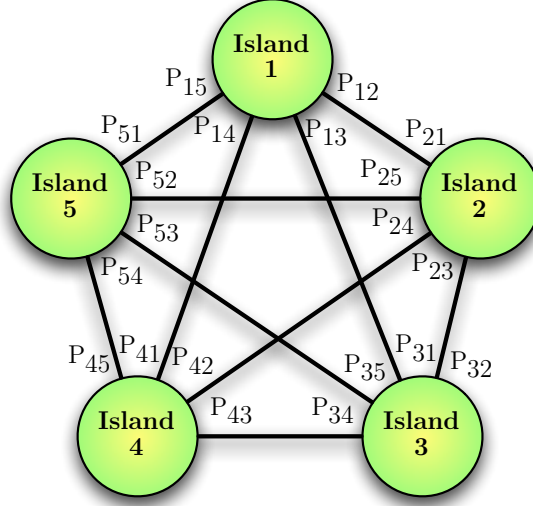


Figure 3: *Island-Model of Distributed Evolutionary Computing: There are different probabilities of going from island 1 to island 2, as there is of going from island 2 to island 1.*

In the coarse-grained *island* models [28, 12], evolution occurs in multiple parallel sub-populations (islands), each running a local evolutionary algorithm, evolving independently with occasional *migrations* of highly fit individuals among sub-populations. The core parameters for the evolutionary algorithm of the island-models are as follows [28]:

- number of the sub-populations: 2, 3, 4, more
- sub-population homogeneity
 - size, crossover rate, mutation rate
- topology of connectivity: ring, star, fully-connected
- static or dynamic connectivity
- migration mechanisms:
 - isolated/synchronous/asynchronous
 - how often migrations occur
 - which individuals migrate

Fine-grained *diffusion* models [44] assign one individual per processor. A local neighbourhood topology is assumed, and individuals are allowed to mate only within their neighbourhood, called a *deme*. The demes overlap by an amount that depends on their shape and size, and in this way create an implicit migration mechanism. Each processor runs an identical evolutionary algorithm which selects parents from the local neighbourhood, produces an offspring, and decides whether to replace the current individual with an offspring. However, even with the advent of multi-processor computers, and more recently multi-core processors, which provide the ability to execute multiple threads simultaneously, this approach would still prove impractical in supporting the number of agents necessary to create a Digital Ecosystem. Therefore, we shall further consider the *island* models.

An example island-model [28, 12] is visualised in Figure 3, in which there are different probabilities of going from island 1 to island 2, as there is of going from island 2 to island